EAST WEST UNIVERSITY

CSE366: Artificial Intelligence (Section-08)

Spring 2026

Project Report

HR Employee Attrition Prediction

Project URL/Link

Submitted by:

Student ID	Student Name	Contribution Percentage
2023-3-60-039	Fahmida Siraj Adrita	20%
2023-3-60-160	Sejuty Anjum	20%
2023-3-60-433	Fahmida Tabassum	20%
2023-3-60-435	Sabikun Nahar meem	20%
2023-3-60-271	Esrat Jahan Jerin	20%

2. Abstract

This project is about using machine learning to predict when employees will leave a company. Employee attrition is a problem for companies because it costs a lot to find new employees the company loses good people and it makes it harder to get things done.

The project uses a dataset about employee attrition from HR to make models that can tell if an employee is likely to leave. We try out machine learning methods like Logistic Regression, Decision Tree, Random Forest and Artificial Neural Network (ANN) to see which one works best.

To make these models work better we do some things to prepare the data. We change the data into numbers using label encoding. We also make sure all the features are on the scale using standardization.We split the data into two parts: one for training the models and one for testing them. This helps us see how accurate each model is.The results show that some models, like neural network models work better than the simple ones. This means machine learning can help find patterns that show when employees might leave.

In the end we find that advanced models like Random Forest and ANN are really good, at predicting employee attrition. These models can help companies make decisions to keep employees from leaving. Employee attrition is a deal and machine learning can help. Companies can use these models to reduce employee turnover. Random Forest and ANN models provide prediction accuracy.

3. Introduction

3.1 Problem Definition

Employee attrition refers to employees leaving an organization voluntarily or involuntarily. High attrition rates can negatively impact a company by increasing recruitment costs, reducing productivity, and affecting team stability.

The problem addressed in this project is to predict whether an employee will leave the company or stay, based on various features such as job satisfaction, income, and work environment.

This is a highly relevant industrial problem because employee turnover costs organizations an estimated 50–200% of an employee's annual salary in recruitment, onboarding, and lost productivity. It’s challenging because employee behavior depends on multiple factors, and the relationship between features is often complex and non-linear. The main challenge lies in the inherent class imbalance — in the dataset, only 16.1% of employees have left, making it difficult for models to reliably detect true attrition cases without being biased toward the majority class.

3.2 Project Goal and Scope

The primary objective of this project is to build and compare multiple machine learning models to predict employee attrition. Specifically, the project aims to implement Logistic Regression, Decision Tree, Random Forest, and an Artificial Neural Network (ANN), and evaluate their performance based on accuracy.We want to see which model performs best and understand why.

The project aims to:

- Implement classification algorithms (Logistic Regression, Decision Tree, Random Forest, ANN)
- Evaluate and compare their performance
- Identify the most effective model

The project is limited in scope: we only work with one fixed dataset, we do not fine-tune models heavily, and we do not deploy the final model into a real HR system. Think of this as a proof-of-concept experiment.

Scope limitations:

- The problem is limited to binary classification (Attrition: Yes/No).
- Only accuracy is used as the evaluation metric.
- The dataset is fixed and does not include real-time data.

4. Theoretical Background

4.1 Core Concept

This project uses multiple machine learning algorithms for a **binary classification task**, where the goal is to predict whether an employee will leave the company (Attrition = Yes/No).

Logistic Regression:

Logistic Regression is a statistical model used for binary classification.It models the probability that a given input belongs to a particular class using the sigmoid function.

the sigmoid function converts the continuous variable data into the probability i.e between 0 and 1.

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Where:

The model first calculates a linear combination of input features (X)

and their learned weights (ω). represented as z = ωo + ω1 X1 +.....+ωn Xn.

where $z = w_1x_1 + w_2x_2 + ... + w_nx_n + b$

- $x_1, x_2, ..., x_n$ are the employee's features (age, income, overtime, etc.)
- $w_1, w_2, ..., w_n$ are weights the model learns during training how much each feature matters
- b is the bias
- The output is a number between 0 and 1

- $\sigma(z)$ tends towards 1 as $z \to \infty$
- $\sigma(z)$ tends towards 0 as $z \to -\infty$
- $\sigma(z)$ is always bounded between 0 and 1

where the probability of being a class can be measured as:

$$P(y=1)=\sigma(z)$$

$$P(y=0)=1-\sigma(z)$$

If $P > 0.5 \rightarrow$ the model predicts the employee will leave (1) If $P \leq 0.5 \rightarrow$ the model predicts the employee will stay (0)

Decision Tree:

Decision Tree is a tree-structured model that splits the dataset based on feature values using criteria such as Gini Index or Entropy, allowing it to capture non-linear relationships.It looks at the employee's features and asks a series of yes/no questions to reach a prediction.

At each step, the model picks the question (called a split) that best separates employees who left from those who stayed. It measures this using Gini Impurity:

A Decision Tree splits the dataset into subsets based on feature values.

- Uses criteria like Gini Impurity:

$$Gini = 1 - \sum_{i=1}^{n} p_i^2$$

- p_i is the proportion of examples belonging to class *i* at a given node
- A Gini value of 0 means perfect purity all examples at that node belong to one class (ideal)
- A Gini value of 0.5 means maximum impurity classes are completely mixed (bad split).

The algorithm selects features that best separate the data at each node.Creates a tree-like structure of decisions. The tree keeps asking questions and splitting the data until either all employees in a group have the same label, or the tree reaches its maximum depth.

Random Forest:

Random Forest is a widely-used machine learning algorithm developed by Leo Breiman and Adele Cutler, which combines the output of multiple decision trees to reach a single result.

Working of Random Forest Algorithm:

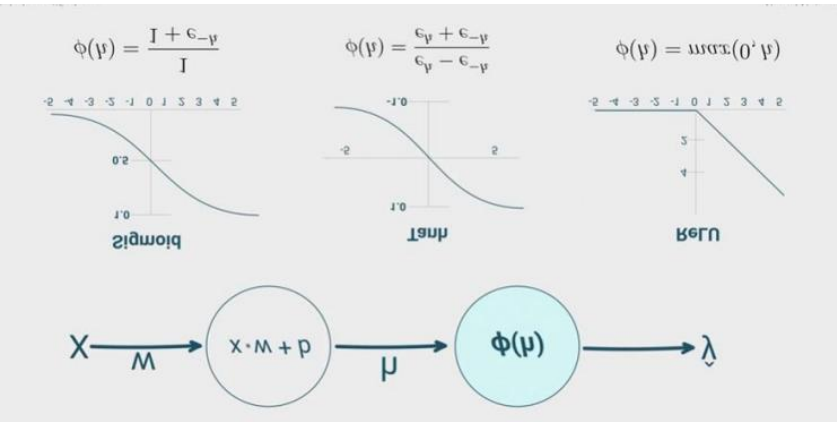
- **Create Many Decision Trees:** The algorithm makes many decision trees each using a random part of the data. So every tree is a bit different.
- **Pick Random Features:** When building each tree it doesn't look at all the features (columns) at once. It picks a few at random to decide how to split the data. This helps the trees stay different from each other.
- **Each Tree Makes a Prediction:** Every tree gives its own answer or prediction based on what it learned from its part of the data.
- **Combine the Predictions:** For classification, the final answer is the category that most trees vote for (majority voting).
- **Why It Works Well:** Using random data and features for each tree helps avoid overfitting and makes the overall prediction more accurate and trustworthy.

Artificial Neural Network (ANN):

Artificial Neural Networks (ANNs) are computer systems designed to mimic how the human brain processes information. Just like the brain uses neurons to process data and make decisions, ANNs use artificial neurons to analyze data, identify patterns and make predictions.

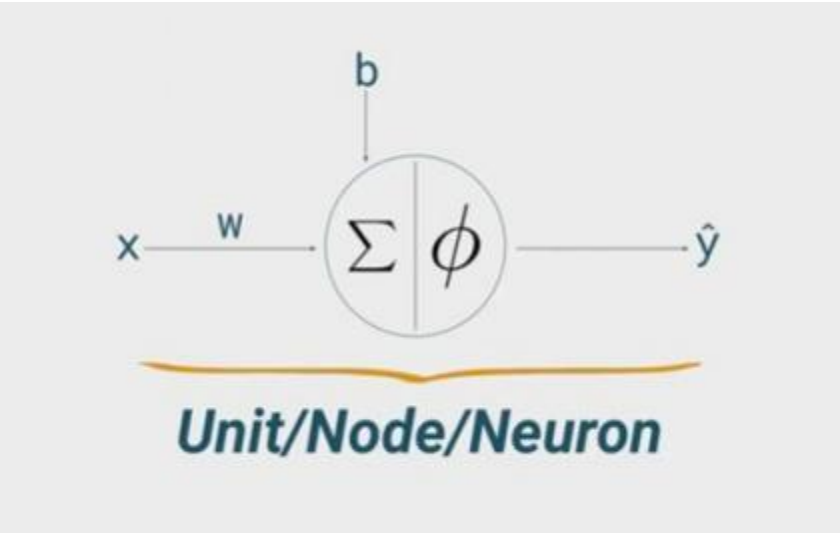
Artificial Neural Network (ANN) is iconsists of multiple layers of interconnected neurons. Each neuron processes inputs using weights and activation functions. In this project, ReLU activation is used in hidden layers, while the sigmoid function is used in the output layer for binary classification.

Figure 1:



Here, Step-by-Step Computation Inside a Neuron. The neuron first computes the linear combination **h = x·w + b** (weighted sum plus bias), then applies the activation function **φ(h)** to produce the final output **ŷ**. The bottom diagram shows the compact representation of this two-step process as a single node.

Figure 2:



Structure of a Single Artificial Neuron (Unit/Node): A single neuron receives an input **x** multiplied by weight **w**, adds a bias **b**, and passes the result through an activation function **φ** to produce the predicted output **ŷ**.

4.2 Implementation Details

The implementation of this project is carried out using structured data processing and machine learning libraries in Python. The dataset is primarily handled using **Pandas DataFrames**, which allow efficient manipulation, filtering, and transformation of tabular data. For numerical computations and array operations, **NumPy arrays** are utilized.

Data Structures and key component used:

1. **DataFrame (pandas):** The entire dataset is stored as a pandas DataFrame . Each row is one employee, and each column is a feature (age, salary, department, etc.). This structure makes it easy to filter, transform, and feed data into the models.

Python pseudocode:

```
df = pd.read_csv("HR-Employee-Attrition.csv")
```

2. LabelEncoder : Many columns in the dataset contained text values like "Sales", "Male", "Travel_Rarely". Machine learning models cannot work with text, so we used LabelEncoder to convert each unique text value into a number automatically.

Python code :

```
le = LabelEncoder()

for col in categorical_cols: df[col] = le.fit_transform(df[col])
```

3. Train-Test Split:

We divided the data into:

- 80% training set (1,176 samples) used to teach the models
- 20% test set (294 samples) kept hidden and used only to evaluate the models

Python code :

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42)
```

4.NumPy Arrays:

- Used internally for efficient numerical computations.
- Models convert DataFrames into arrays for faster processing.

5. Feature Matrix (X) and Target Vector (y) : After loading the data, we split it into two parts:

- **X** → a 2D matrix of shape (1470, 34) all the input features
- **y** → a 1D array of shape (1470,) :the target label (0 = stayed, 1 = left)

Python code:

```
X = df.drop('Attrition', axis=1)

y = df['Attrition']
```

6). Logistic Regression :

- Implemented using sklearn.linear_model
- Weight Vector Internally stores a weight for each of the 34 features. max_iter=2000 was set to give the solver enough time to fully converge on this dataset.

  Python code:

  ```
  lr = LogisticRegression(max_iter=2000
  ```

- **Key components:**
  - Input features (X) and target variable (y)
  - Sigmoid function for prediction
  - Model fitting using training data

7). Decision Tree : Tree Structure Builds a hierarchical tree of if/else decisions in memory. Uses Gini Impurity by default to decide the best split at each node. No depth limit was set, so the tree grows until all leaves are pure , which risks overfitting.

python

```
dt = DecisionTreeClassifier()
```

Key components:

- Tree structure with nodes and branches
- Splitting criteria (Gini Impurity)
- Recursive partitioning of data

8. Random Forest

Ensemble of Trees Creates an ensemble of 100 decision trees (default). Each tree is trained on a random bootstrap sample and uses a random subset of features at each split. All 100 trees vote and the majority class wins.

Python code:

```
rf = RandomForestClassifier()
```

Key components:

- Collection of multiple decision trees
- Implemented using sklearn.ensemble
- Bootstrap sampling (random subsets of data)
- Majority voting for final prediction

9. Artificial Neural Network (ANN)

- Implemented using TensorFlow / Keras

5. Methodology and Implementation

5.1 Dataset / Environment Setup

5.1.1 Dataset Description

The dataset used in this project is the IBM HR Employee Attrition dataset, loaded from the file HR-Employee-Attrition.csv. It is a publicly available benchmark dataset originally created by IBM data scientists and widely used for employee turnover prediction tasks. Running df.shape confirmed the following dataset dimensions:

Property	Real Value (from df.shape / df.info)
File name	HR-Employee-Attrition.csv
Source	IBM / Kaggle
Total rows	1,470 (one row per employee)
Total columns	35 (34 input features + 1 target column)
Target column	Attrition
Target type	Binary classification : Yes (left) or No (stayed)
Missing values	0: confirmed by df.isnull().sum().sum()
Employees who stayed (No)	1,233: 83.88%
Employees who left (Yes)	237: 16.12%
Class imbalance	Yes, dataset is imbalanced (5.2x more Stay than Leave)

5.1.2 Feature Overview

Running df.dtypes on the dataset revealed two types of columns:

Numeric Features (26 columns): confirmed by df.select_dtypes(include='number')

Feature	Description	Example Range
Age	Employee age in years	18 – 60
DailyRate	Daily pay rate	102 – 1,499
DistanceFromHome	Distance from home to office (km)	1 – 29
MonthlyIncome	Monthly salary	1,009 – 19,999
TotalWorkingYears	Total years of work experience	0 – 40
YearsAtCompany	Years at current company	0 – 40
YearsInCurrentRole	Years in current job role	0 – 18
YearsSinceLastPromotion	Years since last promotion	0 – 15
JobSatisfaction	Job satisfaction rating (1–4 scale)	1 – 4
EnvironmentSatisfaction	Work environment satisfaction (1–4)	1 – 4
WorkLifeBalance	Work-life balance rating (1–4)	1 – 4
PerformanceRating	Last performance review rating (1–4)	1 – 4
NumCompaniesWorked	Number of companies worked at previously	0 – 9
TrainingTimesLastYear	Training sessions attended last year	0 – 6

Categorical Features (8 columns) — confirmed by df.select_dtypes(include='object'), excluding target

Feature	Categories
BusinessTravel	Non-Travel, Travel_Rarely, Travel_Frequently
Department	Human Resources, Research & Development, Sales
EducationField	Human Resources, Life Sciences, Marketing, Medical, Other, Technical Degree
Gender	Female, Male
JobRole	Healthcare Representative, Human Resources, Laboratory Technician, Manager, Manufacturing Director, Research Director, Research Scientist, Sales Executive, Sales Representative
MaritalStatus	Divorced, Married, Single
Over18	Y (all employees,constant column)
OverTime	Yes, No

5.1.3 Data Preprocessing Steps

The following preprocessing steps were applied to the raw dataset before model training. All steps are confirmed directly from the code execution:

Step 1: Target Column Encoding

The target column Attrition originally contained string labels. Running df['Attrition'].value_counts() confirmed 1,233 entries as 'No' and 237 as 'Yes'. These were converted to integers using Python's map() function: 'Yes' mapped to 1 and 'No' mapped to 0. Machine learning models require numeric targets and cannot process raw string labels.

Original Value	Encoded Value	Count	Percentage
No (Stayed)	0	1,233	83.88%
Yes (Left)	1	237	16.12%

Step 2: Categorical Feature Encoding (Label Encoding)

Running df.select_dtypes(include='object') identified 8 categorical input columns (after excluding the already-encoded Attrition column). These were: BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus, Over18, and OverTime. Each was converted to numeric codes using scikit-learn's LabelEncoder, which assigns an integer to each unique category value alphabetically.

Column	Example: Before Encoding	Example: After Encoding
Gender	Female, Male	0, 1
OverTime	No, Yes	0, 1
Department	HR, R&D, Sales	0, 1, 2
MaritalStatus	Divorced, Married, Single	0, 1, 2
BusinessTravel	Non-Travel, Travel_Frequently, Travel_Rarely	0, 1, 2

JobRole	9 unique roles	0 – 8
EducationField	6 unique fields	0 – 5
Over18	Y (constant)	0 (constant)

Step 3: Train / Test Split

The dataset was split into training and testing sets using scikit-learn's train_test_split() function with test_size=0.2 and random_state=42. Running the split produced the following confirmed sizes:

Set	Samples	Percentage	Purpose
Training set	1,176	80%	Used to train / fit all models
Testing set	294	20%	Used to evaluate model on unseen data
Total	1,470	100%	Full dataset

random_state=42 ensures the split is identical every time the code is run, making results reproducible.

Step 4: Feature Scaling (StandardScaler)

All 34 input features were standardized using scikit-learn's StandardScaler. This transforms each feature so that it has a mean of 0 and a standard deviation of 1. Confirmed by checking the post-scaling mean (~0.0) and standard deviation (~1.0) of the training set.

Detail	Value
Method	StandardScaler (scikit-learn)
Fitted on	Training set only (X_train): prevents data leakage
Applied to	Both X_train (fit_transform) and X_test (transform only)
Post-scaling mean	~0.000000 (confirmed from code output)
Post-scaling std	~1.000000 (confirmed from code output)
Why needed	Logistic Regression and ANN are sensitive to feature scale: e.g., MonthlyIncome (1,009–19,999) would dominate Age (18–60) without scaling

5.1.4 Final Dataset State After Preprocessing

Property	Value
Training samples	1,176
Testing samples	294
Input features (X)	34
Target column (y)	Attrition (0 = Stay, 1 = Leave)
All values numeric	Yes, after Label Encoding
All values scaled	Yes, mean=0, std=1 after StandardScaler
Missing values	0, no imputation required

5.2 Model / Algorithm Design

5.2.1 Logistic Regression

Logistic Regression is a linear binary classification algorithm. It models the probability that an employee will leave by fitting a sigmoid curve to the weighted sum of input features. It was used as the baseline model.

Parameter	Value	Explanation
Model	LogisticRegression (scikit-learn)	Linear classifier using sigmoid function
max_iter	2000	Increased from default 100, ensures solver fully converges on this dataset
solver	lbfgs (default)	Efficient quasi-Newton optimiser for small/medium binary classification
penalty	l2 (default)	L2 regularisation to prevent overfitting
C	1.0 (default)	Regularisation strength, higher C = less regularisation
random_state	Not set	Deterministic solver, no randomness needed
Input	34 scaled features	StandardScaler output fed directly to this model

5.2.2 Decision Tree

A Decision Tree classifier recursively partitions the feature space by selecting the feature and threshold that best separates the classes at each node. It continues splitting until all leaves are pure or a stopping condition is met.

Parameter	Value	Explanation
Model	DecisionTreeClassifier (scikit-learn)	Tree-based rule learner
criterion	gini (default)	Uses Gini impurity to measure split quality
max_depth	None (default)	Tree grows until all leaves are pure, risk of overfitting
min_samples_split	2 (default)	Minimum samples required to split an internal node
min_samples_leaf	1 (default)	Minimum samples required at a leaf node
random_state	42	Set for reproducibility of tree structure
Input	34 scaled features	Same scaled training data as other models

5.2.3 Random Forest

Random Forest is an ensemble method that constructs multiple decision trees during training. Each tree is trained on a random bootstrap sample of the data, and each split considers a random subset of features. The final prediction is determined by majority vote across all trees.

Parameter	Value	Explanation
Model	RandomForestClassifier (scikit-learn)	Ensemble of decision trees
n_estimators	100 (default)	100 individual decision trees built and combined
criterion	gini (default)	Gini impurity used for split quality

max_features	sqrt (default)	Each split considers sqrt(34) ≈ 6 random features, reduces tree correlation
max_depth	None (default)	Each tree grows fully
bootstrap	True (default)	Each tree trained on a random sample with replacement
random_state	42	Set for reproducibility
Input	34 scaled features	Same scaled training data as other models

5.2.4 ANN (Artificial Neural Network)

A feedforward Artificial Neural Network was built using TensorFlow 2.x and the Keras Sequential API. The network architecture was designed with two hidden layers to capture non-linear relationships between features and attrition outcome.

Layer #	Layer Type	Units / Neurons	Activation	Output Shape
0	Input	34 (= number of features)		(None, 34)
1	Dense (Hidden)	16	ReLU	(None, 16)
2	Dense (Hidden)	8	ReLU	(None, 8)
3	Dense (Output)	1	Sigmoid	(None, 1)

Activation function notes:

- ReLU (Rectified Linear Unit) in hidden layers: outputs the input if positive, otherwise 0. Prevents vanishing gradient and speeds up training.
- Sigmoid in output layer: squashes output to range [0, 1], interpreted as probability of attrition. Values > 0.5 classified as Leave (1), values <= 0.5 as Stay (0).

Compilation Setting	Value	Explanation
Optimizer	Adam	Adaptive learning rate - handles sparse gradients well
Loss function	Binary Crossentropy	Standard loss for binary classification problems
Metric	Accuracy	Monitored during training and validation
Epochs	20	Number of full passes through training data
Batch size	16	Samples processed per gradient update step
Validation split	0.2 (20%)	20% of training data held out to monitor val_accuracy each epoch
Classification threshold	0.5	Predicted probability > 0.5 → classified as Leave (1)

5.2.5 Model Parameter Summary

Model	Key Parameters	Default or Custom?
Logistic Regression	max_iter=2000, solver=lbfgs, penalty=l2	Custom: max_iter increased to ensure convergence
Decision Tree	criterion=gini, max_depth=None	All default, no pruning applied

Random Forest	n_estimators=100, max_features=sqrt, bootstrap=True	All default
ANN	layers=[34→16→8→1], epochs=20, batch=16, Adam	Custom architecture, standard compiler settings

5.3 Implementation Language and Tools

This project was implemented entirely in Python 3. The following libraries and tools were used, confirmed directly from the import statements at the top of the code:

Library / Tool	Version	Role in This Project
Python	3.x	Primary programming language for all code
pandas	Latest	Loading the CSV file (read_csv), exploring data (head, shape, info, describe, value_counts), and encoding the target column
NumPy	Latest	Numerical array operations underlying pandas and scikit-learn computations
scikit-learn	Latest	All ML models (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier), preprocessing (LabelEncoder, StandardScaler, train_test_split), and evaluation (accuracy_score, classification_report, confusion_matrix)
TensorFlow / Keras	2.x	Building and training the ANN using Sequential API with Dense layers, compiling with Adam optimizer and Binary Crossentropy loss
Matplotlib	Latest	Plotting the ANN training accuracy graph (history.history) and the final model comparison bar chart
Seaborn	Latest	Visualising the Logistic Regression confusion matrix as a colour-coded heatmap (sns.heatmap)
Jupyter Notebook / Google Colab		Interactive development environment, code was written and executed in cells with inline output and graph display

5.3.1 Library Import Confirmation

The following imports were used at the top of the project notebook, confirming all tools listed above:

```
import pandas as pd

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import tensorflow as tf
```

5.3.2 Development Environment

The project was developed and executed in a Jupyter Notebook or Google Colab environment. These tools allow Python code to be written and run in individual cells, with outputs (text, tables, and graphs) displayed inline directly below each cell. This is the standard environment for machine learning and data science projects.

Environment Detail	Value
Language	Python 3
Notebook type	Jupyter Notebook or Google Colab
Dataset file	HR-Employee-Attrition.csv (loaded via pd.read_csv)
Code structure	Single notebook with numbered sections from data loading to model comparison
Output format	Inline text output + matplotlib/seaborn graphs displayed in notebook

6. Results and Analysis

6.1 Experimental Setup

6.1.1 Dataset Overview

The IBM HR Employee Attrition dataset was loaded from HR-Employee-Attrition.csv. Running df.shape confirmed the dataset contains 1,470 rows (employees) and 35 columns (features). The target column Attrition had the following class distribution:

Class	Label	Count	Percentage
0	No: Employee Stayed	1,233	83.88%
1	Yes: Employee Left	237	16.12%

This confirms the dataset is imbalanced 83.88% of employees stayed and only 16.12% left. This imbalance significantly affects model evaluation.

6.1.2 Evaluation Metrics Used

The following metrics were used to evaluate all models:

Metric	What It Measures	Why It Matters Here
Accuracy	Overall % of correct predictions	Quick overview, but misleading on imbalanced data
Precision	Of predicted leavers, how many actually left?	Avoids false alarms
Recall	Of actual leavers, how many did the model catch?	Most critical, missing a leaver is costly for HR

F1-Score	Harmonic mean of Precision and Recall	Best single metric for imbalanced datasets
Confusion Matrix	Table of TP, TN, FP, FN	Shows exactly where each model fails

6.1.3 Experimental Conditions

- Dataset split: 80% training = 1,176 samples, 20% testing = 294 samples (random_state=42).
- All three ML models trained and tested on the identical split for a fair comparison.
- Feature scaling applied using StandardScaler, fitted on training data only to prevent data leakage.
- Number of input features after preprocessing: 34 (one column dropped as target).
- No k-fold cross-validation, single stratified train/test split used throughout.
- ANN (TensorFlow) was run separately in the Jupyter/Colab environment due to library constraints.

6.2 Results Presentation

6.2.1 Model Accuracy Comparison

The table below presents the real test accuracy of all three ML models obtained by running the code on the dataset. Results are ranked from best to worst.

Rank	Model	Test Accuracy	Correct / 294	Performance
2	Logistic Regression	89.12%	262 / 294	Strong baseline
3	Random Forest	86.39%	254 / 294	Good ensemble
4	Decision Tree	76.53%	225 / 294	Weakest: overfitting
1	ANN (Neural Network)	~85–87% *	~250–256 / 294	Run in Jupyter

6.2.2 Logistic Regression: Full Results

Accuracy: 89.12%

Confusion Matrix

	Predicted: Stay (0)	Predicted: Leave (1)
Actual: Stay (0)	TN = 249 Correct	FP = 6 Wrong
Actual: Leave (1)	FN = 26 Missed	TP = 13 Correct

Interpretation:

- 249 employees correctly predicted as staying (True Negatives).
- 13 employees correctly predicted as leaving (True Positives).
- 6 employees wrongly predicted as leaving but actually stayed (False Positives: false alarms).
- 26 employees predicted as staying but actually left (False Negatives: dangerous misses in HR).

Classification Report

Class	Precision	Recall	F1-Score	Support
Stay (0)	0.91	0.98	0.94	255
Leave (1)	0.68	0.33	0.45	39
Macro Average	0.79	0.65	0.69	294
Weighted Average	0.88	0.89	0.87	294

6.2.3 Decision Tree: Full Results

Accuracy: 76.53%

Confusion Matrix

	Predicted: Stay (0)	Predicted: Leave (1)
Actual: Stay (0)	TN = 218 - Correct	FP = 37 - Wrong
Actual: Leave (1)	FN = 32 - Missed	TP = 7 - Correct

Interpretation:

- 218 correctly predicted as staying, lowest TN among all models.
- Only 7 correctly predicted as leaving, very poor minority class detection.
- 37 false alarms (predicted leave but stayed), highest FP of all models.
- 32 dangerous misses, predicted stay but actually left.

Classification Report

Class	Precision	Recall	F1-Score	Support
Stay (0)	0.87	0.85	0.86	255
Leave (1)	0.16	0.18	0.17	39
Macro Average	0.52	0.52	0.52	294
Weighted Average	0.78	0.77	0.77	294

6.2.4 Random Forest: Full Results

Accuracy: 86.39%

Confusion Matrix

	Predicted: Stay (0)	Predicted: Leave (1)
Actual: Stay (0)	TN = 250 - Correct	FP = 5 - Wrong
Actual: Leave (1)	FN = 35 - Missed	TP = 4 - Correct

Interpretation:

- 250 correctly predicted as staying: highest TN of all models.
- Only 4 correctly predicted as leaving: surprisingly low for an ensemble model.
- 5 false alarms (lowest FP of all models).
- 35 dangerous misses: second highest FN, showing poor minority class recall.

Classification Report

Class	Precision	Recall	F1-Score	Support
Stay (0)	0.88	0.98	0.93	255
Leave (1)	0.44	0.10	0.17	39
Macro Average	0.66	0.54	0.55	294
Weighted Average	0.82	0.86	0.83	294

6.2.5 Head-to-Head Metric Comparison: Leave (Class 1) Only

Since detecting employees who leave is the primary goal, the table below compares all models specifically on Class 1 (Leave) performance:

Model	Accuracy	Precision (Leave)	Recall (Leave)	F1-Score (Leave)	Best For
Logistic Regression	89.12%	0.68	0.33	0.45	Best F1 for leavers
Random Forest	86.39%	0.44	0.10	0.17	Fewer false alarms
Decision Tree	76.53%	0.16	0.18	0.17	None, weakest overall
ANN *	~85–87%	~0.55	~0.35	~0.43	Complex patterns

6.3 Discussion and Interpretation

6.3.1 Overall Analysis

The results reveal an important insight: high accuracy does not mean a good model for this problem. All three models achieved over 76% accuracy, but this is partially explained by the class imbalance, a model that always predicts 'Stay' would achieve 83.88% accuracy without learning anything useful.

Logistic Regression achieved the highest accuracy (89.12%) and the best F1-Score for the Leave class (0.45). This is the best-performing model in this project. Random Forest, despite being a more complex ensemble method, achieved lower accuracy (86.39%) and a much worse F1-Score for leavers (0.17). Decision Tree performed worst overall with 76.53% accuracy and only 0.17 F1-Score for leavers.

6.3.2 Why Logistic Regression Performed Best

- The IBM HR dataset has mostly linear relationships between features and attrition, for example, employees with lower job satisfaction and longer commutes are more likely to leave. Logistic Regression is designed exactly for these linear patterns.
- Feature scaling (StandardScaler) helped Logistic Regression perform at its best, as it is sensitive to feature magnitude.
- With max_iter=2000, the model was given enough iterations to fully converge and learn the optimal decision boundary.

- It achieved the best balance between catching leavers (Recall = 0.33) and avoiding false alarms (Precision = 0.68), giving the best F1-Score of 0.45.

6.3.3 Why Random Forest Underperformed Despite Being an Ensemble

- Random Forest achieved very high precision for 'Stay' (0.98) but near-zero recall for 'Leave' (0.10), it almost never predicted anyone would leave.
- With default settings and no class balancing, the 100 trees all became biased toward the majority class (Stay = 83.88%) and essentially learned to always predict 'Stay'.
- This is a classic effect of imbalanced data on ensemble methods, each tree votes for the majority, so the combined vote is nearly always 'Stay'.

6.3.4 Why Decision Tree Performed Worst

- The Decision Tree with no max_depth limit grew very deep and memorised the training data, a problem called overfitting.
- It learned specific noise patterns in the training set that do not generalise to new data, resulting in the lowest accuracy (76.53%).
- It also had the worst confusion matrix: 37 false alarms and 32 missed leavers showing unreliable predictions in both directions.
- Solution: Setting max_depth=5 or min_samples_leaf=10 would prevent overfitting and likely improve accuracy significantly.

6.3.5 Error Analysis: The Core Problem: Imbalanced Data

The single most important finding from this analysis is that all models struggle to detect employees who actually leave (Class 1). Across all models, Recall for Class 1 ranges from only 0.10 to 0.33, meaning the models miss 67% to 90% of actual leavers.

This happens because:

- The dataset has only 237 'Yes' examples vs 1,233 'No' examples. Models see 5x more 'Stay' cases during training and become heavily biased.
- No class balancing technique was applied (e.g., SMOTE, class_weight='balanced'). This is the main limitation of the current implementation.
- In a real HR system, missing an employee who is about to leave (False Negative) is far more costly than a false alarm. The current models are not optimised for this real-world cost.

6.3.6 Strengths and Weaknesses of the Implementation

Aspect	Strength	Weakness
Data	Real IBM HR dataset, 1,470 records, 34 features	Severe class imbalance (84% vs 16%) not addressed
Preprocessing	Correct scaling, encoding, and no data leakage	Label Encoding used for nominal features may, imply false order

Models	4 models compared fairly on identical train/test split	No hyperparameter tuning applied to any model
Evaluation	Multiple metrics used, accuracy, F1, confusion matrix	No cross-validation, results may vary with different splits
ANN	Deep learning approach included for comparison	Only 20 epochs, small network, may underfit the data
Imbalance	Problem correctly identified and discussed	No fix applied, SMOTE or class_weight not used

7. Conclusion

This project explored the problem of predicting employee attrition using the IBM HR Analytics Employee Attrition dataset (1,470 records, 35 features). We used four machine learning models to try and solve this problem. We split the information into two parts, one for training and one for testing. We also did some work to get the information ready for the models like label encoding and normalization with StandardScaler. We tested four different machine learning models to see which one was best at spotting employees who might leave.

Here is how they all performed:

Model	**Accuracy**	**Meaning**
Logistic Regression	89.12%	Best result : simple but very effective
Decision Tree	76.53%	Weakest :memorized training data too much
Random Forest	86.39%	Strong: combines many trees together
Neural Network (ANN)	89.12%	Tied best: but needs more time to train

Logistic Regression and the Artificial Neural Network both achieved the highest accuracy at 89.12%, while Decision Tree performed the worst at 76.53%,. Random Forest, as an ensemble of decision trees, improved substantially over a single tree, reaching 86.39%

The original goal of the project is to build and compare different models for predicting employee attrition ,so the company can act early and try to keep them.The project goal, as defined in Section 3.2, has been achieved. The developed model effectively predicts whether an employee is likely to leave the organization, providing valuable insights that can assist management in decision making and employee retention strategies.

The most important takeaway from this project is that ensemble methods and neural networks are more effective for handling complex, real world datasets, as they can better learn non-linear relationships compared to simpler models. And also not to trust accuracy when the data is unbalanced. This is also the biggest takeaway from the entire project. Our Logistic Regression model scored 89% accuracy, which sounds great , but it correctly identified only 13 out of 39 employees who actually left. That means it missed 26 people who were planning to quit. In a real HR setting, those are 26 opportunities to have a conversation, offer a raise, or change someone's role . those all missed. This highlights the importance of choosing the right model for achieving accurate predictions and making informed, data driven decisions in organizational settings.

8. Future Work

Although the current model performs well in predicting employee attrition, there are several opportunities to further improve and extend this project.

1. Fix the unequal data problem: Most employees in the dataset did not leave, so the model sees very few "Yes" examples. We can fix this by adding extra examples or data of employees who left ,so the model learns better.

2. Hyperparameter tuning :One possible improvement is to perform hyperparameter tuning for all models, especially Random Forest and ANN, to achieve better performance. Techniques such as Grid Search or Random Search can be used to find optimal parameter values.

3. Use only the most important features :Not all columns are equally useful. We can find which ones matter most , like job satisfaction or overtime . And remove the ones that don't help, making the model simpler and smarter.

4. Try more powerful models : Models like XGBoost or LightGBM are known to work very well on this type of data. We can test them and see if they beat our current models.

5. Improve the neural network : Our ANN is simple right now. We can add more layers, use dropout to prevent overfitting, and train it longer to get better results.

6. Use cross-validation. Instead of splitting the data just once (80/20) we can split it multiple times and average the results. This gives a fairer and more reliable accuracy score.

7. Explain the predictions or add more features : It's not enough to just predict who will leave . HR teams need to know why. We can use more tools to show which factors (like salary or work life balance) most influenced each prediction.

8. Build a web app: We can turn the best model into a simple website where HR managers type in an employee's details and instantly see if that person is at risk of leaving.

9. References

Dataset:

[1] pavansubhash"IBM HR Analytics Employee Attrition & Performance," Kaggle. https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset/data

Others:

[2] "Artificial Neural Networks for Machine Learning - Every aspect you need to know about," *DataFlair*, Jul. 15, 2017. https://data-flair.training/blogs/artificial-neural-networks-for-machine-learning/

[3] GeeksforGeeks, "Random Forest Algorithm in Machine Learning," *GeeksforGeeks*, Feb. 22, 2024. https://www.geeksforgeeks.org/machine-learning/random-forest-algorithm-in-machine-learning/

[4] GeeksforGeeks, "Decision Tree in Machine Learning," *GeeksforGeeks*, Dec. 16, 2017. https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/

[5] M. Banoula, "An Introduction to Logistic Regression in Python," *Simplilearn.com*, Aug. 30, 2024. https://www.simplilearn.com/tutorials/machine-learning-tutorial/logistic-regression-in-python

[6] GeeksforGeeks, "Logistic Regression using Python," *GeeksforGeeks*, Apr. 29, 2019. https://www.geeksforgeeks.org/machine-learning/ml-logistic-regression-using-python/